Error Handling with Exceptions

How can programs go wrong?

The program is used incorrectly:

- User wants to open a file, but the file does not exist.
- User wants to connect to a domain, but the domain does not exist.
- User wants to perform an action, but does not have permission.

The program depends on unreliable resources:

- The network is down.
- The file system is full.
- No more memory is available.

The program violates a protocol or performs nonsensical operations:

- Allocated memory is never deallocated.
- Deallocated memory is still accessed.
- Tries to divide by zero.

How can we handle errors?

Incorrect use of the program:

- Explain to the user what is wrong and ask for a corrective action!
- Recoverable, we can ask the user for help.

Unreliable resources:

- If the network is down, we can **try again later**.
- If the file system is full, we can **ask for corrective action**.
- Often recoverable. Perhaps the problem will fix itself or the user can help.

Protocol Violations

- Programming bugs: we, as the developers, made a mistake.
- Almost always unrecoverable, the program itself must be **corrected**.
- Not much to do, other than to try to save data, and exit the program.

How can we discover an error?

We need a **mechanism** to report when something has gone wrong.

Typical scenario: We call a function/method/procedure. Did it go well?

- **A Global Variable holds an error code, if any.**
  - Commonly used in C. Old school. Avoid.
- **Return an error code, if any. Otherwise return a success value.**
  - Commonly used in C, but also in many other languages.
  - What if we actually wanted to return a value? Sometimes -1 for error, and rest for the result.
- **Return a data structure which encodes success/failure.**
  - Commonly used in functional languages, e.g. Scala, Haskell, Ocaml, etc.
- **Redirect control-flow with an *exception*.**
  - Commonly used in Java, C#, and current industry languages.
  - What we will talk about for the remainder.

Exceptions

A control-flow mechanism to handle abnormal situations.

- We **throw** and **catch** exceptions.
- An exception is an object (an instance of a class).

We **throw** an exception when we want to abort execution in a method.

- A throw statement immediately abort execution, similar to a return.
- A return statement only exits the *current* method, but a throw statement *continues* exiting methods until the exception is caught.

We **catch** an exception when we want to handle it.

- Execution continues from where the exception is caught.
- If an exception is never caught, it reaches main, and the program aborts.
- The part that catches an exception is called an *exception handler*.

Throwing Exceptions

```
public class BankAccount {

    private int balance;

    public void withdraw(int amount) {
        if (amount <= 0) {
            String msg = "Amount must be positive.";
            throw new IllegalArgumentException(msg);
        }

        if (amount >= balance) {
            String msg = "Amount must be less than balance.";
            throw new IllegalArgumentException(msg);
        }

        balance = balance - amount;
    }

}
```

Try-Catch

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    BankAccount account = new BankAccount(100);

    while (true) {
        System.out.println("Enter an amount to withdraw:");
        int amount = scanner.nextInt();

        try {
            account.withdraw(amount);
        } catch (IllegalArgumentException e) {
            System.out.println(“Something went wrong.");
        }
    }
}
```

Custom Exceptions

```
public class NegativeAmountException extends RuntimeException {

    public NegativeAmountException() { }

}

public class InsufficientFundsException extends RuntimeException {

    private int deficit;

    public InsufficientFundsException(int deficit) {
        this.deficit = deficit;
    }

    public int getDeficit() {
        return deficit;
    }

}
```

Throwing Exceptions

```
public class BankAccount {

    private int balance;

    public void withdraw(int amount) {
        if (amount <= 0) {
            throw new NegativeAmountException();
        }

        if (amount >= balance) {
            int deficit = amount - balance;
            throw new InsufficientFundsException(deficit);
        }

        balance = balance - amount;
    }

}
```

Catching Multiple Exceptions

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    BankAccount account = new BankAccount(100);

    while (true) {
        System.out.println("Enter an amount to withdraw:");
        int amount = scanner.nextInt();

        try {
            account.withdraw(amount);
        } catch (NegativeAmountException e) {
            System.out.println("Amount cannot be negative!");
        } catch (InsufficientFundsException e) {
            System.out.println("You are short: " + e.getDeficit());
        }
    }
}
```

Uncaught Exceptions

Uncaught exceptions propagate to **main** and abort the program.

Every exception includes a **stack trace** of the methods that were currently on the call stack when the exception occurred.

```
Exception in thread "main" NegativeAmountException
     at BankAccount.withdraw(BankAccount.java:11)
     at Main.main(Main.java:13)
```

Real stack traces are typically much longer!

- Try a Google search for “long stack trace”.

Checked and Unchecked

In Java exceptions come in two variants:

- A **checked** exception must be caught or re-thrown.
  - A subclass of `Exception` is a checked exception.
- An **unchecked** exception may be caught or re-thrown.
  - A subclass of `RuntimeException` is an unchecked exception.

The checked exception controversy:

- Programmers dislike being told what to do.
- Painful to use library with lots of checked exceptions.
- Widely considered to be a design mistake.
- Consequently, checked exceptions omitted from C# and C++.

Checked Exceptions

```
public class InsufficientFundsException extends Exception {

    // ...

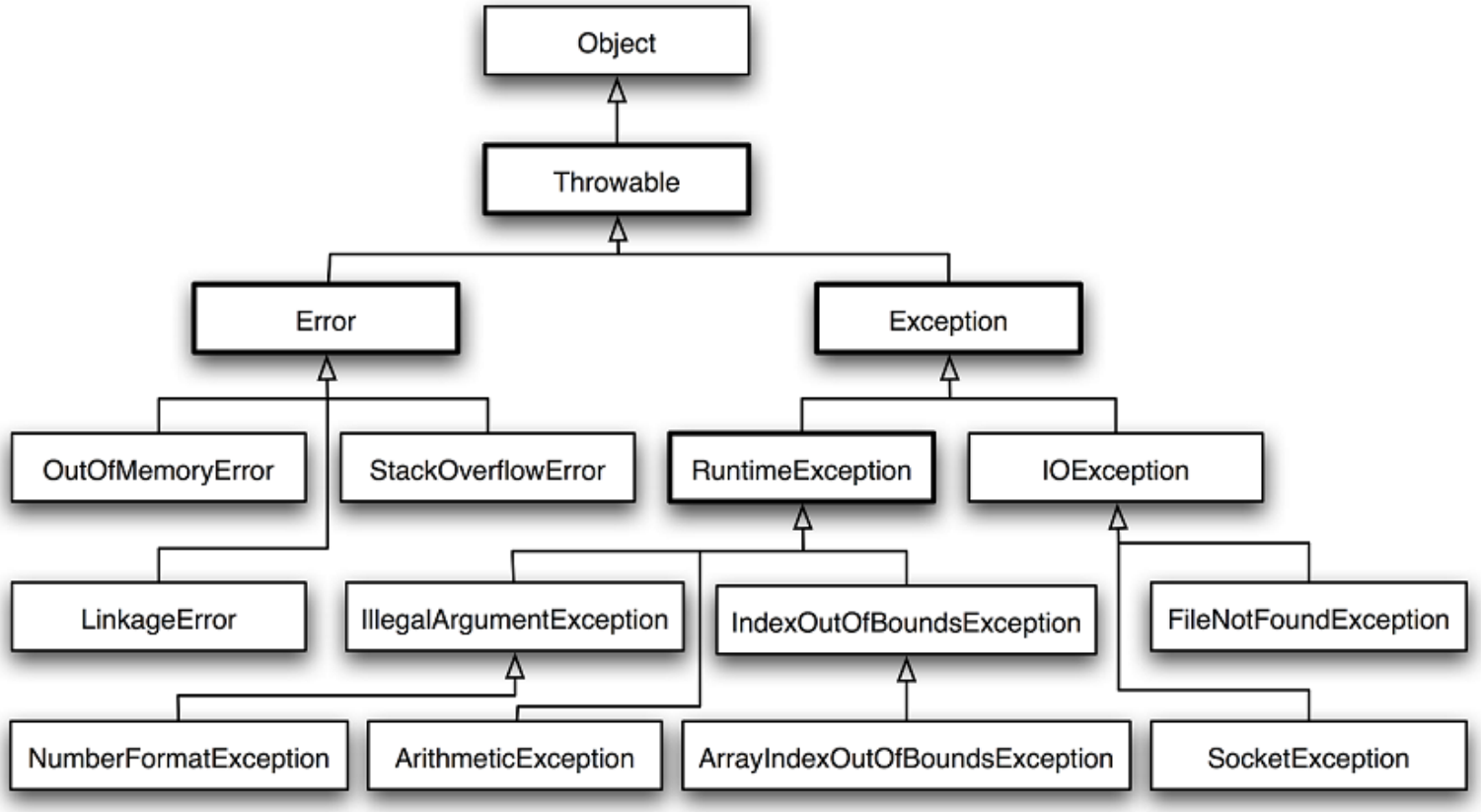
}
```

```
public void withdraw(int amount) throws InsufficientFundsException {
    // ...

    if (amount >= balance) {
        int deficit = amount - balance;
        throw new InsufficientFundsException(deficit);
    }

    // ...
}
```

Exception Hierarchy



(image source: https://itblackbelt.wordpress.com/2015/02/17/checked-vs-unchecked-exception-in-java-example/)